

DataPilot AI: System Architecture Blueprint & Technical Specification

1. System Architectural Vision & Core Paradigm Shift

1.1 Paradigm Shift: Conversational Chatbots vs. Executable Data Workflows

Context & Strategic Importance

The rapid adoption of Artificial Intelligence in enterprise analytics has highlighted a critical operational bottleneck: simple conversational AI text generators are fundamentally unsuited for rigorous, production-grade data engineering and decision support. Modern enterprises require a strategic shift away from unstructured, non-deterministic text generation toward governed, agentic data engineering and analytics platforms. DataPilot AI addresses this challenge by redefining the boundary between generative intelligence and deterministic execution, converting plain-English user intent into structured, reproducible, and verifiable data pipelines.

Analytical Transformation

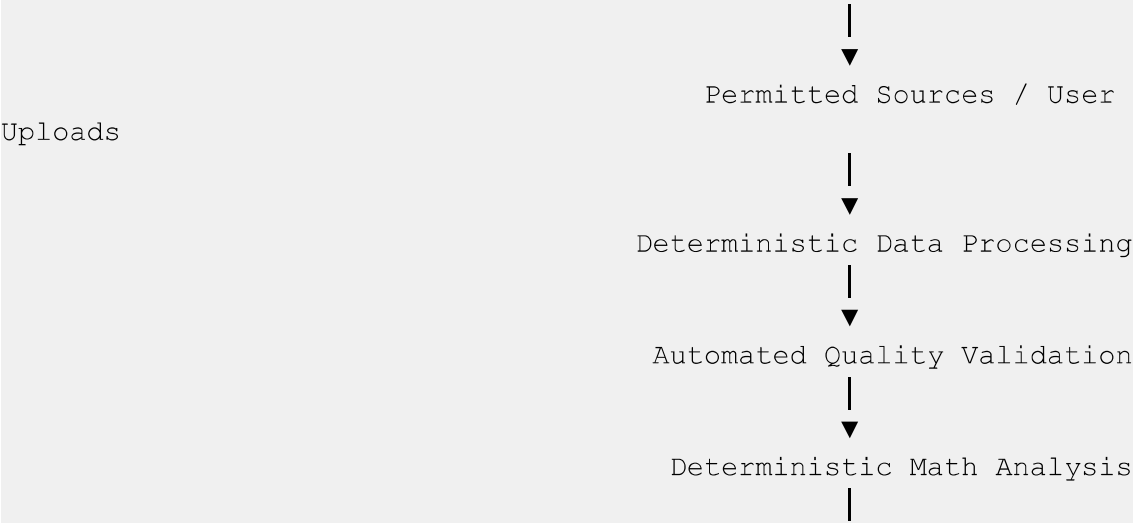
Standard AI chatbots operate on a simplistic, single-stage interaction model (Question → LLM → Answer). In this model, the Large Language Model (LLM) is improperly tasked with interpreting intent, retrieving data, calculating metrics, and rendering final answers simultaneously. This monolithic pipeline inherently introduces non-deterministic outputs, mathematical hallucinations, and severe security vulnerabilities. DataPilot AI replaces this pattern with a multi-stage, orchestrated pipeline: $\text{Question} \rightarrow \text{Intent} \rightarrow \text{Plan} \rightarrow \text{Collect} \rightarrow \text{Process} \rightarrow \text{Validate} \rightarrow \text{Analyze} \rightarrow \text{Explain} \rightarrow \text{Evidence}$

Standard Chatbot Pattern:

User Question → Large Language Model → Unverified Text Answer

DataPilot AI Paradigmatic Pattern:

User Question → Intent Understanding → Dynamic Workflow Generation



▼

Visualization & Insight Generation → Provenance & Lineage Trail

A primary vulnerability in traditional AI analytics is relying on basic Text-to-SQL approaches. Simple Text-to-SQL engines fail under real-world semantic ambiguity. For instance, if a database schema contains columns such as `sales_amount`, `amount`, `revenue`, `net_revenue`, and `gross_sales`, a naive Text-to-SQL engine must guess which field corresponds to a generic user query like "Show me sales." Similarly, entity variations (e.g., NYC, New York, New York City, NY) and temporal representations (2025, FY2025, 2024-25) frequently cause database queries to yield inaccurate or corrupted statistical aggregations. DataPilot AI resolves semantic ambiguity through a governed **Semantic Layer**. The Semantic Layer abstracts underlying raw database fields into business logic definitions (e.g., defining Revenue as `SUM(selling_price)` or Profit as `selling_price - cost_price`). The mapping configuration explicitly bridges natural language concepts with underlying data structures:

```
{
  "semantic_layer": {
    "metrics": {
      "revenue": {
        "description": "Total gross monetary value of units sold",
        "sql_expression": "SUM(selling_price)",
        "underlying_columns": ["selling_price"]
      },
      "profit": {
        "description": "Net monetary gain after deducting unit cost",
        "sql_expression": "SUM(selling_price - cost_price)",
        "underlying_columns": ["selling_price", "cost_price"]
      },
      "fastest_growing": {
        "description": "Entity ranking based on Compound Annual Growth Rate",
        "math_expression": "((value_end / value_start) ^ (1 / n)) - 1",
        "default_operation": "CAGR"
      }
    },
    "entities": {
      "manufacturer": {
        "canonical_field": "manufacturer_name",
        "synonyms": ["brand", "maker", "company", "oem"]
      }
    }
  }
}
```

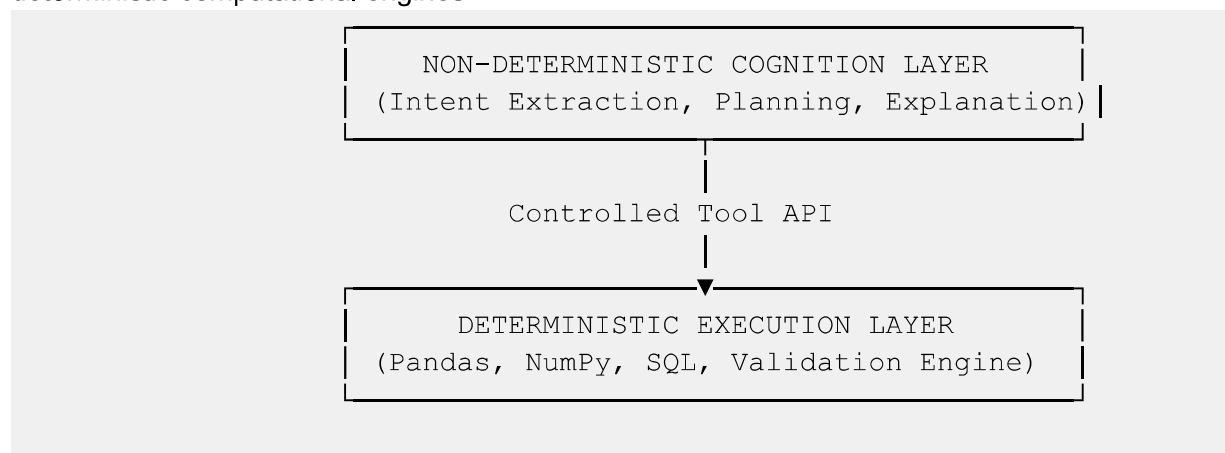
When a user submits a query, the platform maps natural language input directly to explicit dimensions, metrics, timeframes, and operations before generating execution steps. | Architectural Dimension | Traditional Data Analytics | Standard AI Chatbots | DataPilot AI Intelligence Paradigm || ----- | ----- | ----- | ----- || **Execution Mechanics** | Manual SQL/Python scripting, manual data cleaning, and static dashboard design. | Single-pass text generation (LLM → Text Response). | AI-orchestrated dynamic workflow generation with sandboxed deterministic tool execution. || **Deterministic Reliability** | High calculation accuracy; high human overhead and operational latency. | Low to Moderate; prone to mathematical hallucinations, silent calculation errors, and context drift. | Complete; mathematical/statistical engine strictly isolated from LLM text generation. || **Governance & Provenance** | Centralized, manual schema governance; rigid data structures. | Absent; black-box generation with zero dataset lineage or step auditability. | Fully governed; automated source validation, schema tracking, explicit validation scoring, and full audit trails. |

Converting an arbitrary natural-language information request into a governed, executable, and auditable data workflow represents a foundational shift in enterprise data intelligence.

1.2 Architectural Principles & Core System Goals

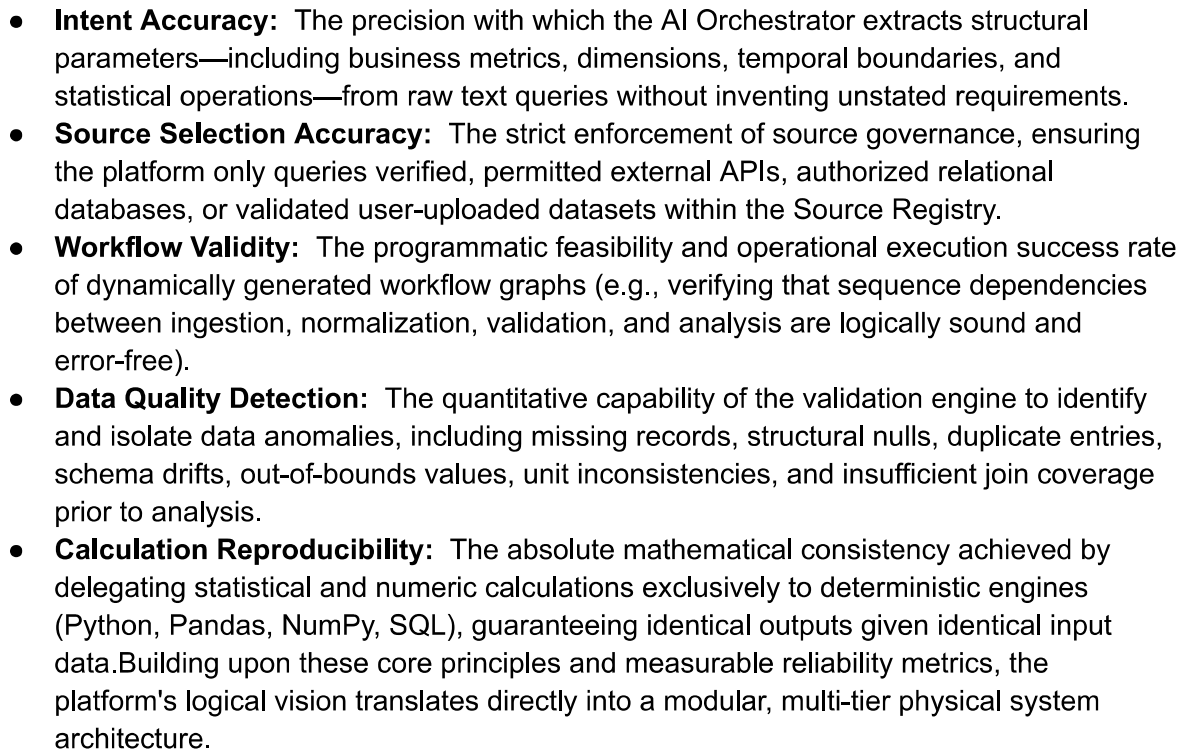
Context & Strategic Importance

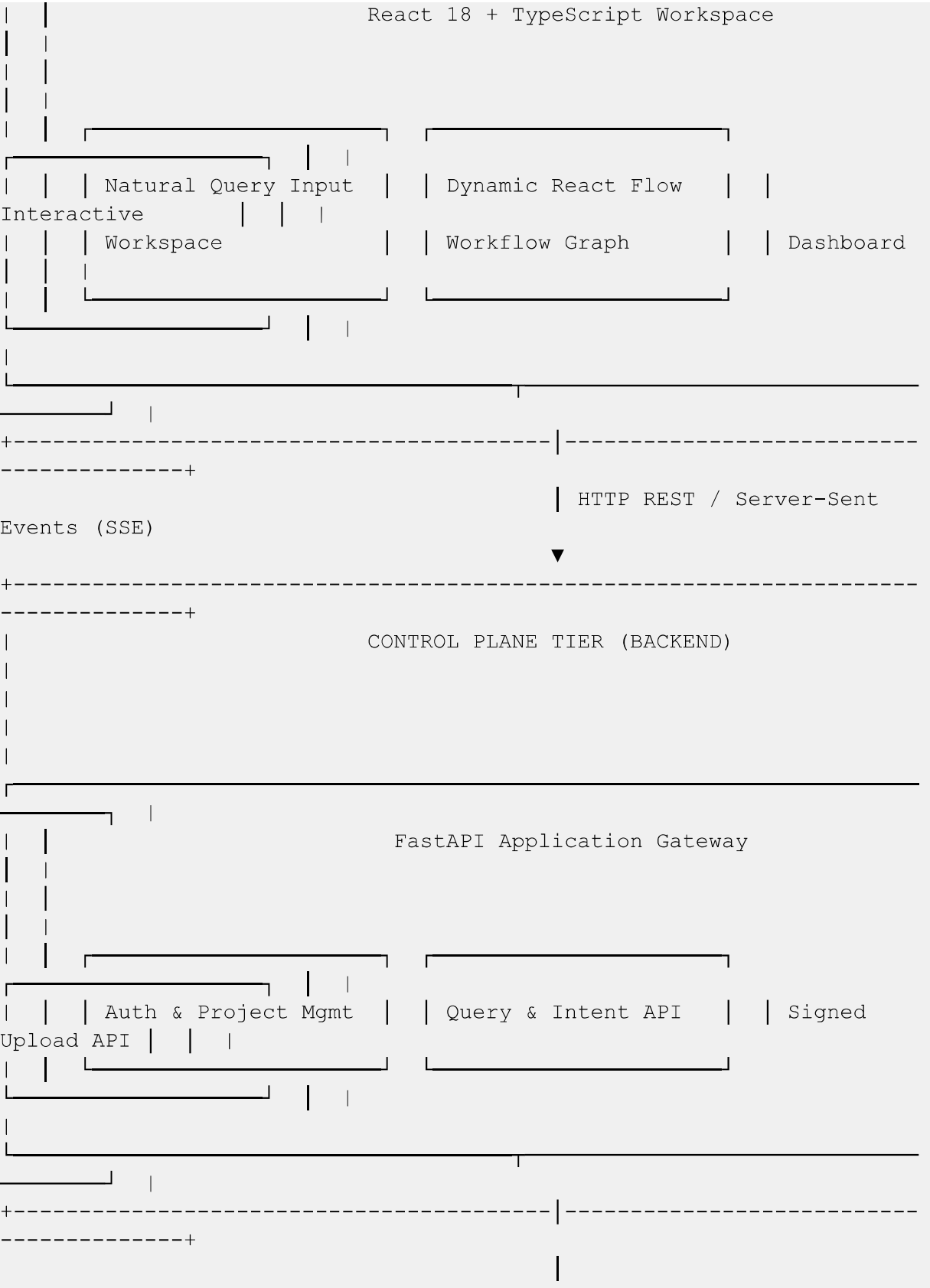
To guarantee enterprise reliability, DataPilot AI is designed around foundational engineering principles: strict isolation of non-deterministic LLMs from numeric processing, total system transparency, and complete lineage auditability. The platform operates on the core thesis that language models should serve strictly as cognition, planning, and contextual explanation layers, leaving numerical calculations, data transformations, and structural aggregations to deterministic computational engines.

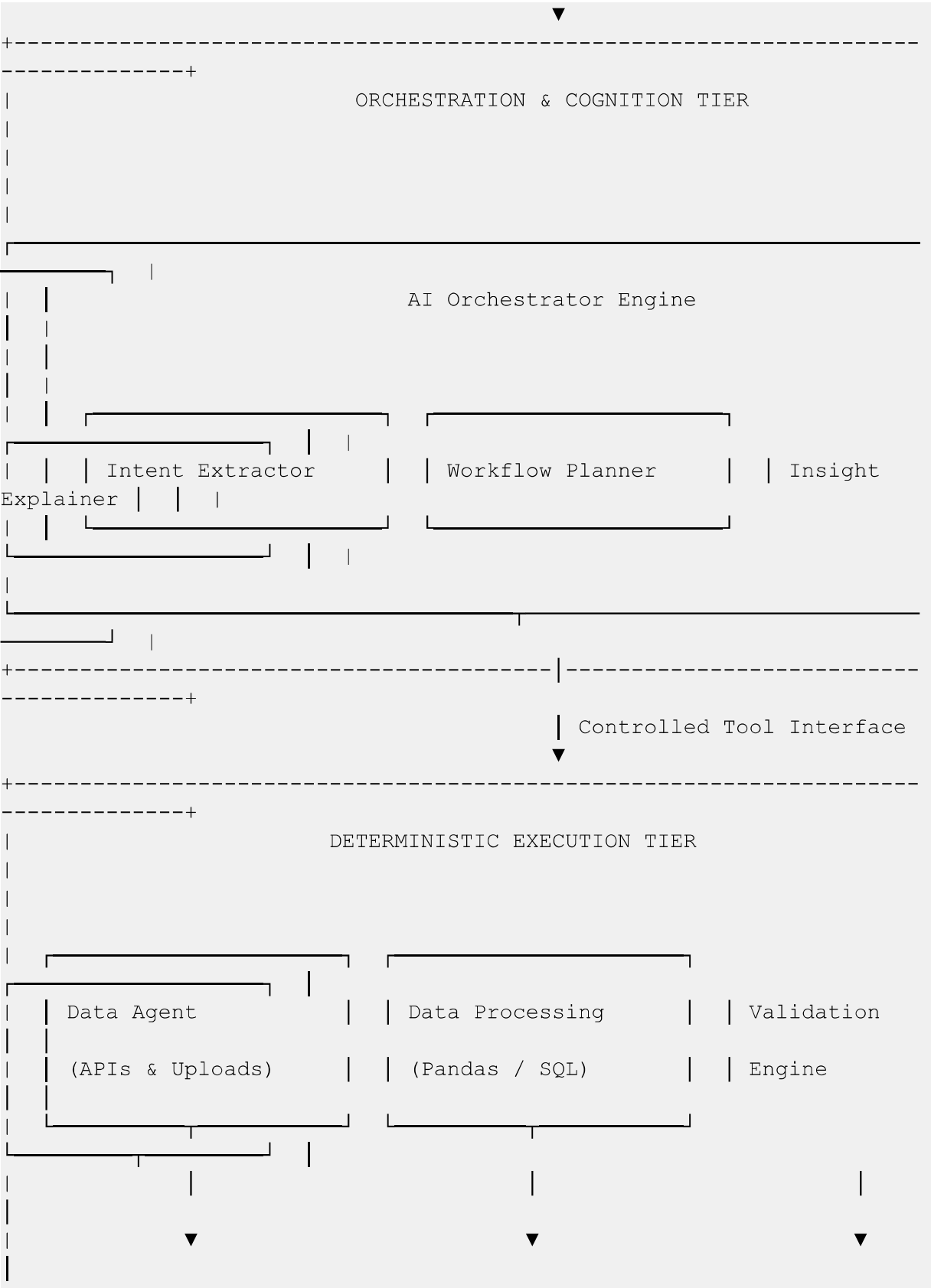


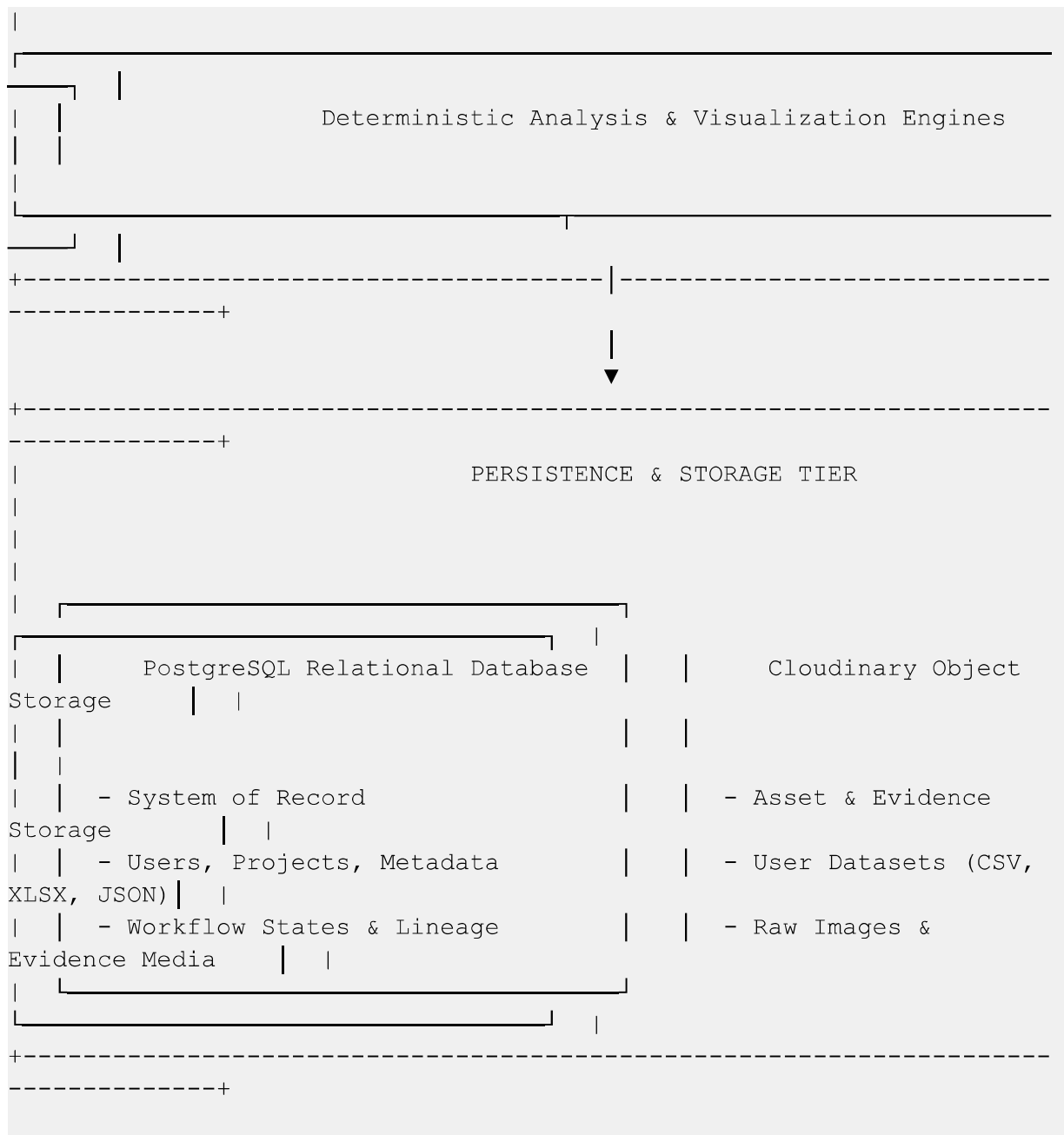
Analytical Transformation

The central architectural objective of DataPilot AI is to convert an ambiguous natural-language information request into a governed, executable, and auditable data workflow. To measure, verify, and sustain platform reliability, the architecture enforces five core evaluation metrics:









Deconstructing the operational responsibilities across the three primary layers reveals strict boundary separations:

1. **Frontend Workspace (React/TypeScript):** Serves as the user's primary intelligence interaction environment. It renders the natural language query interface, dynamically displays interactive execution flow charts using React Flow, streams real-time step progress using Server-Sent Events (SSE), and renders final dashboards using custom chart components (Recharts/ECharts).
2. **Control Plane API (FastAPI):** Acts as the system central gatekeeper. Built on FastAPI, Pydantic, and SQLAlchemy, this asynchronous Python layer manages authentication,

project session isolation, query ingestion, Cloudinary upload signature generation, workflow execution state tracking, and schema validation.

3. **Execution Engine (Python/Pandas/NumPy):** Executes deterministic data processing functions. Restricted strictly to sandboxed operations, this tier executes source data fetching, string/entity normalization, missing value handling, schema checking, numerical calculations, trend aggregations, statistical correlation analyses, and chart schema construction.

2.2 Technology Stack & Module Organization

Context & Strategic Importance

Selection of the underlying technology stack prioritizes performance, type safety, deterministic processing execution, and robust enterprise scalability. Integrating Python's data engineering suite with React's component framework provides a clean foundation for handling asynchronous workflow streaming and mathematical workloads.

Analytical Transformation

The following mapping specifies the technology stack components paired directly with their architectural duties: | Technology Component | Precise Architectural Function || ----- | ----- || **React 18 & TypeScript** | Client-side user interface rendering, type-safe application state management, dynamic UI rendering. || **Vite** | Frontend build system, asset bundling, and developer workflow optimization. || **Tailwind CSS** | Styling engine for building user workspace layouts and modular data components. || **React Flow** | Interactive node-based graph framework for visualizing dynamic workflows and lineage connections. || **Recharts / ECharts** | Declarative chart rendering library for dynamic visualization output (line, bar, scatter, maps). || **Lucide React** | System UI iconography suite for consistent visual indicators across workflow statuses. || **FastAPI** | High-performance Python asynchronous backend framework serving REST APIs and real-time SSE endpoints. || **Pydantic** | Strict server-side data validation, query payload parsing, and structured intent schema enforcement. || **SQLAlchemy** | Object-Relational Mapping (ORM) framework managing state interactions with PostgreSQL. || **Pandas & NumPy** | Deterministic data cleaning, entity normalization, schema joins, array processing, and statistical analytics. || **PostgreSQL** | Primary relational system of record storing metadata, workflow states, validation metrics, and lineage records. || **Cloudinary** | Asset and evidence storage management storing uploaded datasets, generated evidence, and reports. || **LangGraph / Custom Engine** | Dynamic state machine managing agent execution graphs, state updates, conditional edges, and error loops. |

To enforce separation of concerns, the enterprise codebase structure organizes agents, processing tools, storage services, and API controllers into clear directory modules:

```
datapilot-ai/  
├── frontend/  
│   ├── src/  
│   └── api/ # REST and SSE client connection  
modules
```

			— components/	# UI elements (QueryInput,
			WorkflowGraph, Dashboard, etc.)	
			— hooks/	# Custom React hooks for workspace
			state and streaming	
			— pages/	# Workspace views (Home, Workspace,
			Datasets, Evidence)	
			— types/	# TypeScript interfaces and backend
			type definitions	
			— App.tsx	# Root application router and
			configuration	
			— package.json	
			— backend/	
			— app/	
			— agents/	# AI Orchestration & Tool Agents
			— data_agent.py	# Ingestion and API collection agent
			— insight.py	# Natural language insight
			generation agent	
			— orchestrator.py	# Intent extraction and workflow
			planning engine	
			— research.py	# Domain and permitted source
			discovery agent	
			— api/	# FastAPI router endpoints
			— auth.py	# Registration and JWT session
			endpoints	
			— datasets.py	# Dataset metadata and upload
			signature routers	
			— evidence.py	# Audit trail and provenance
			endpoints	
			— projects.py	# User project workspace isolation
			routers	
			— queries.py	# Query submission and initial
			processing API	
			— workflows.py	# Real-time SSE streaming and
			workflow execution APIs	
			— engines/	# Deterministic execution mechanisms
			— analysis.py	# Statistical and mathematical
			aggregation engine	
			— processing.py	# Pandas cleaning, filtering, and
			normalization engine	
			— validation.py	# Data health scoring and quality
			checking engine	
			— visualization.py	# JSON chart configuration generator
			— models/	# SQLAlchemy ORM database models
			— schemas/	# Pydantic data validation contracts

```

|   |   |— services/           # External infrastructural
interfaces
|   |   |   |— cloudinary.py    # Cloudinary asset storage
integration service
|   |   |   |— database.py      # PostgreSQL pool and session
manager
|   |   |   |— source_registry.py # Approved API & DB source
permissions engine
|   |   |   |— main.py          # FastAPI application initialization
entry point
|   |   |— docker-compose.yml
|   |   |— requirements.txt
|— README.md

```

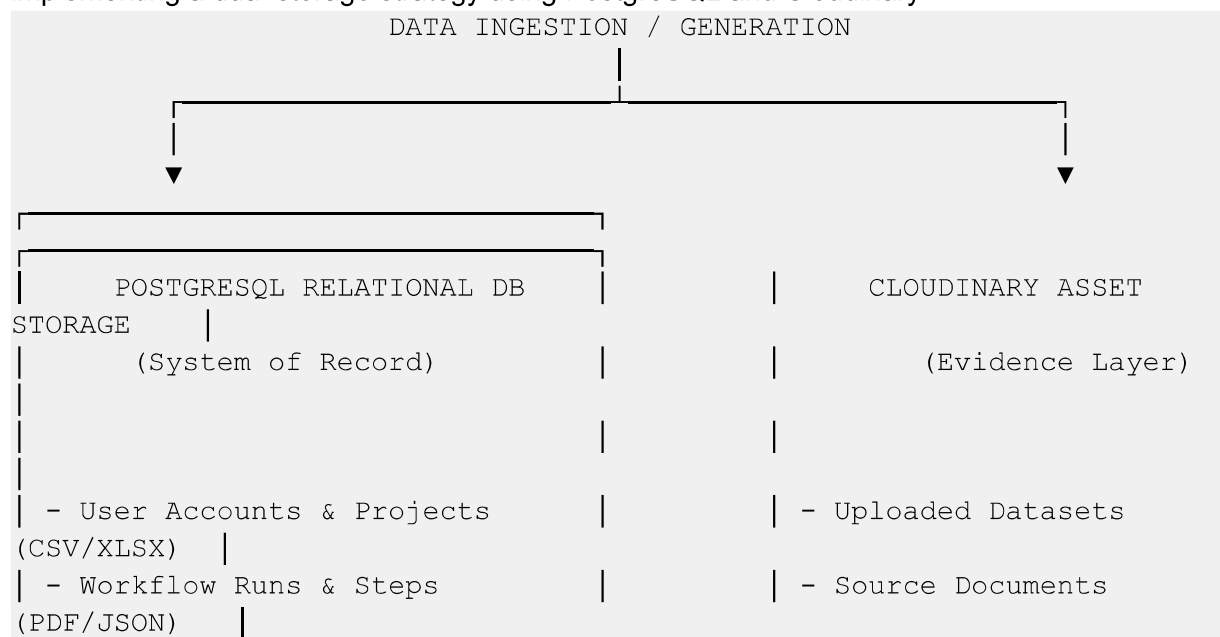
This clear structural separation guides how state and operational artifacts are persisted across the platform's dual-storage strategy.

3. Dual-Storage Infrastructure Strategy: PostgreSQL vs. Cloudinary

3.1 Division of Responsibilities: System of Record vs. Evidence Layer

Context & Strategic Importance

A core architectural requirement of DataPilot AI is separating low-latency transactional system state from heavy binary assets and static evidence artifacts. Mixing raw dataset payloads (e.g., multi-megabyte Excel or CSV files) and generated report binaries into a relational database degrades query throughput and complicates scalability. DataPilot AI addresses this by implementing a dual-storage strategy using PostgreSQL and Cloudinary.



- Validation Run Scores		- Generated Reports & Visuals
- Evidence Lineage References		- Immutable Evidence Artifacts

Analytical Transformation

PostgreSQL serves as the **System of Record** , managing all structured metadata, relational hierarchies, application state, user identities, query histories, data schemas, validation scores, and execution steps. Conversely, Cloudinary functions as the **Asset, Evidence, and Provenance Layer** , storing raw input datasets, source evidence documents, exported report visual assets, and generated media artifacts.

Data Category	Targeted Storage System	Specific Artifacts Stored	Architectural Rationale for Storage Allocation
---------------	-------------------------	---------------------------	--

|| ----- || ----- || ----- || ----- ||

User & Project Auth | PostgreSQL | User IDs, password hashes, email, project names, descriptions, timestamps. | Relational integrity, low-latency querying, transactional consistency, strict user multi-tenancy. ||

Dataset Assets | Cloudinary | Raw .csv, .xlsx, .json, .pdf uploaded data files. | Bypasses API server bottlenecks via direct upload; offers scalable binary storage and reliable CDN distribution. ||

Dataset Metadata | PostgreSQL | Cloudinary Public IDs, asset URLs, schema structures, record counts, dataset health scores. | Enables fast relational querying without loading heavy binary dataset payloads into memory. ||

Workflow State | PostgreSQL | Active query blueprints, workflow step sequences, runtime status, execution timestamps, error logs. | Requires high transactional reliability, state lock capabilities, and low-latency update operations. ||

Quality & Validation | PostgreSQL | Completeness percentages, duplicate counts, out-of-bounds metrics, schema evaluation flags. | Structured numeric metrics used directly in programmatic UI rendering and system confidence scoring. ||

Evidence & Lineage | PostgreSQL & Cloudinary | Lineage mapping keys (PostgreSQL) referencing underlying source files and media proofs (Cloudinary). | Connects structured database audit trails directly with immutable raw assets managed in object storage. |

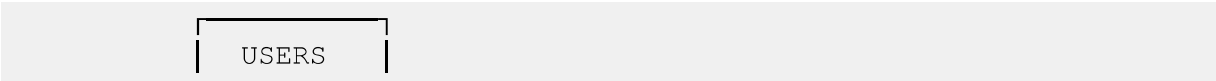
3.2 PostgreSQL Relational Database Schema Design

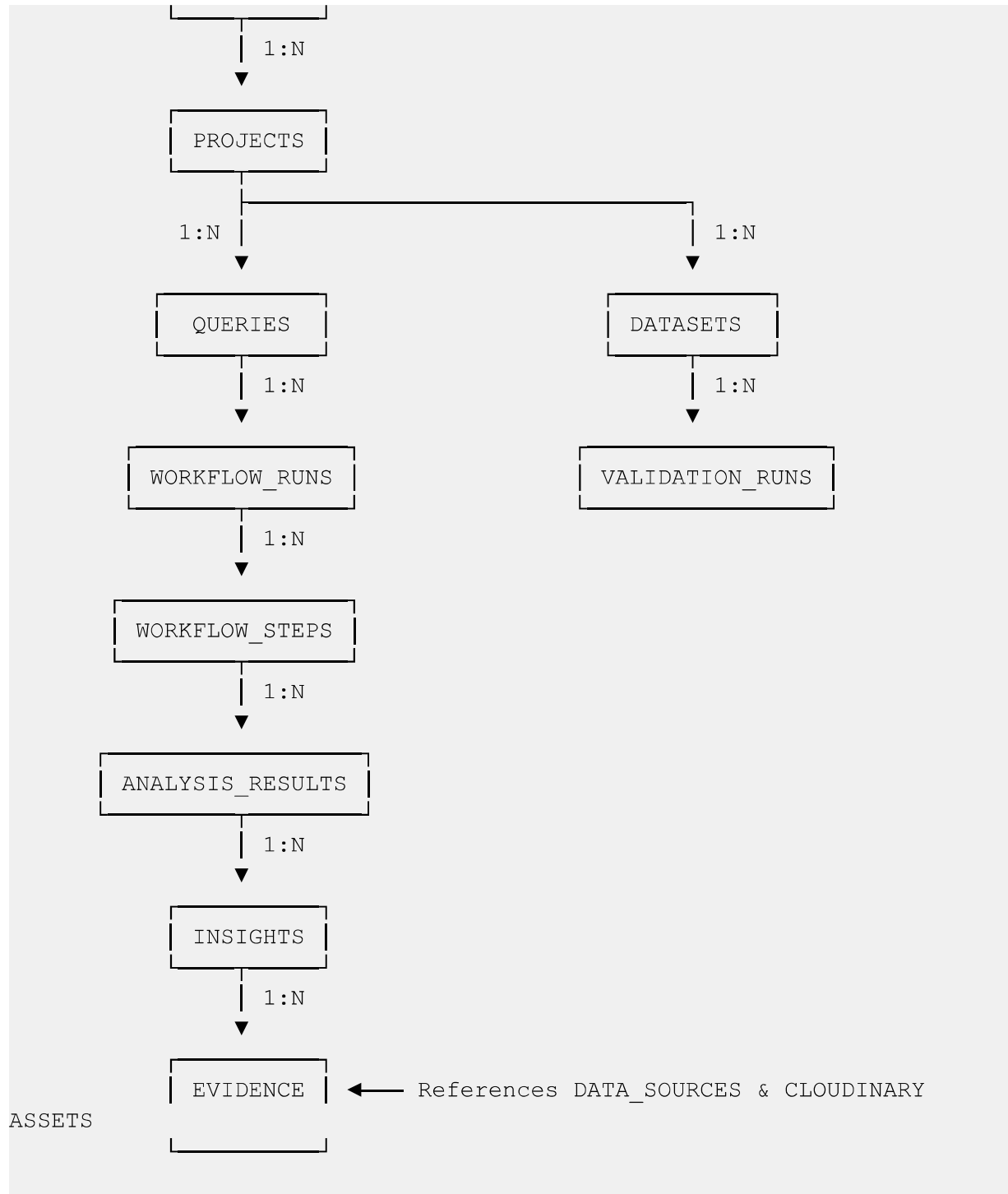
Context & Strategic Importance

Supporting dynamically generated multi-step analytics requires a production-grade relational schema. The database model must enforce referential integrity and maintain low-latency query performance across user workspace abstractions, workflow steps, validation metrics, deterministic calculation outputs, generated AI insights, and granular lineage trace keys.

Analytical Transformation

The logical dependency hierarchy traces query execution from user accounts down to evidence artifacts:





The enterprise physical schema specification is defined using production DDL standards:

```

-- Core User & Project Isolation Models
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,

```

```

        password_hash VARCHAR(255) NOT NULL,
        created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL
    );

CREATE TABLE projects (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL
);

-- Query Ingestion and Intent Blueprints
CREATE TABLE queries (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE
CASCADE,
    natural_language_query TEXT NOT NULL,
    structured_intent JSONB NOT NULL,
    status VARCHAR(50) NOT NULL CHECK (status IN ('PENDING',
'PROCESSING', 'COMPLETED', 'FAILED', 'NEEDS_CLARIFICATION')),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL
);

-- Source Governance & Data Ingestion Metadata
CREATE TABLE data_sources (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    type VARCHAR(50) NOT NULL CHECK (type IN ('api', 'user_upload',
'database')),
    base_url VARCHAR(1024),
    description TEXT,
    allowed BOOLEAN DEFAULT TRUE NOT NULL,
    license VARCHAR(255) NOT NULL,
    domain VARCHAR(255) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL
);

CREATE TABLE datasets (

```

```

        id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        project_id UUID NOT NULL REFERENCES projects(id) ON DELETE
CASCADE,
        name VARCHAR(255) NOT NULL,
        source_type VARCHAR(50) NOT NULL,
        cloudinary_public_id VARCHAR(512) NOT NULL,
        cloudinary_url VARCHAR(1024) NOT NULL,
        file_type VARCHAR(20) NOT NULL,
        schema JSONB NOT NULL,
        record_count INTEGER NOT NULL DEFAULT 0,
        quality_score NUMERIC(5,2) CHECK (quality_score >= 0 AND
quality_score <= 100),
        created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL
    );

-- Orchestration Workflow Tracking
CREATE TABLE workflow_runs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    query_id UUID NOT NULL REFERENCES queries(id) ON DELETE CASCADE,
    status VARCHAR(50) NOT NULL CHECK (status IN ('PLANNING',
'RUNNING', 'COMPLETED', 'FAILED')),
    started_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL,
    completed_at TIMESTAMP WITH TIME ZONE

);

CREATE TABLE workflow_steps (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workflow_id UUID NOT NULL REFERENCES workflow_runs(id) ON DELETE
CASCADE,
    step_number INTEGER NOT NULL,
    step_type VARCHAR(50) NOT NULL CHECK (step_type IN ('intent',
'source_discovery', 'cleaning', 'validation', 'analysis',
'visualization')),
    input JSONB NOT NULL,
    output JSONB,
    status VARCHAR(50) NOT NULL CHECK (status IN ('PENDING',
'RUNNING', 'COMPLETED', 'FAILED', 'SKIPPED')),
    started_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL,
    completed_at TIMESTAMP WITH TIME ZONE,
    error TEXT
);

```

```

-- Validation Engine & Quantitative Metrics
CREATE TABLE validation_runs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    dataset_id UUID NOT NULL REFERENCES datasets(id) ON DELETE
CASCADE,
    workflow_id UUID NOT NULL REFERENCES workflow_runs(id) ON DELETE
CASCADE,
    completeness NUMERIC(5,2) NOT NULL CHECK (completeness >= 0 AND
completeness <= 100),
    duplicate_count INTEGER NOT NULL DEFAULT 0,
    invalid_count INTEGER NOT NULL DEFAULT 0,
    join_coverage NUMERIC(5,2) CHECK (join_coverage >= 0 AND
join_coverage <= 100),
    schema_valid BOOLEAN NOT NULL DEFAULT TRUE,
    validation_details JSONB NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL
);

-- Analytics Output & AI Insight Persistence
CREATE TABLE analysis_results (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workflow_id UUID NOT NULL REFERENCES workflow_runs(id) ON DELETE
CASCADE,
    analysis_type VARCHAR(100) NOT NULL,
    result JSONB NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL
);

CREATE TABLE insights (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workflow_id UUID NOT NULL REFERENCES workflow_runs(id) ON DELETE
CASCADE,
    title VARCHAR(255) NOT NULL,
    description TEXT NOT NULL,
    calculation TEXT NOT NULL,
    confidence NUMERIC(5,2) NOT NULL CHECK (confidence >= 0 AND
confidence <= 100),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL
);

CREATE TABLE evidence (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

        insight_id UUID NOT NULL REFERENCES insights(id) ON DELETE
CASCADE,
        dataset_id UUID REFERENCES datasets(id) ON DELETE SET NULL,
        workflow_step_id UUID NOT NULL REFERENCES workflow_steps(id) ON
DELETE CASCADE,
        source_id UUID REFERENCES data_sources(id) ON DELETE SET NULL,
        transformation TEXT NOT NULL,
        calculation TEXT NOT NULL,
        source_reference VARCHAR(1024) NOT NULL,
        created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL
    );

-- Strategic Indexes for Low-Latency Query Execution
CREATE INDEX idx_queries_project ON queries(project_id);
CREATE INDEX idx_queries_intent_gin ON queries USING GIN
(structured_intent);
CREATE INDEX idx_datasets_project ON datasets(project_id);
CREATE INDEX idx_workflow_steps_workflow ON
workflow_steps(workflow_id);
CREATE INDEX idx_workflow_steps_input_gin ON workflow_steps USING GIN
(input);
CREATE INDEX idx_validation_runs_dataset ON
validation_runs(dataset_id);
CREATE INDEX idx_evidence_insight ON evidence(insight_id);

```

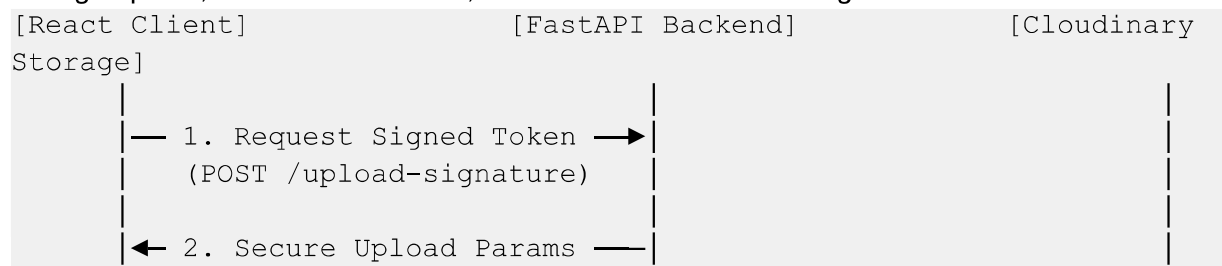
3.3 Cloudinary Asset Ingestion & Evidence Flow

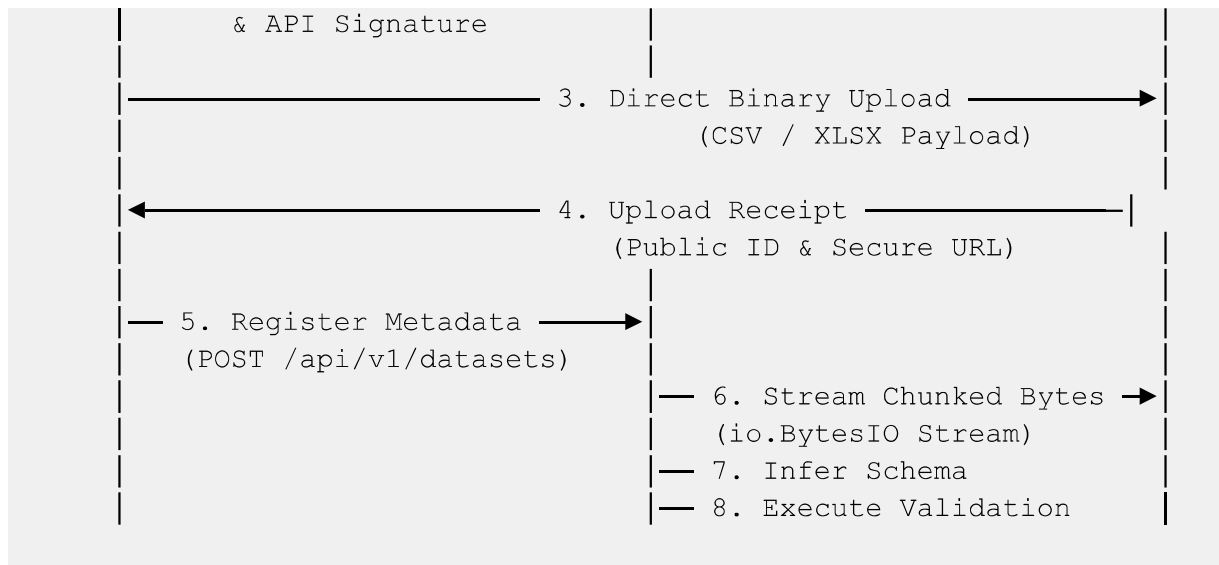
Context & Strategic Importance

To prevent backend API memory saturation during large dataset uploads, DataPilot AI utilizes a direct-to-cloud upload paradigm. Secure parameter signing enables client instances to stream files directly to Cloudinary storage. This ensures backend resources remain dedicated to orchestration, validation, and analytics rather than binary file transfer.

Analytical Transformation

The direct ingestion flow processes incoming user files through explicit backend authentication, storage upload, metadata extraction, and automatic validation stages:





1. **Authentication & Token Generation:** The React frontend initiates the upload process by sending a request to the FastAPI control plane endpoint (POST /api/v1/datasets/upload-signature).
2. **Signature Allocation:** FastAPI validates the user's authentication credentials, constructs a time-bound upload payload, signs it using server-side Cloudinary credentials, and returns the upload parameters to the client.
3. **Direct Cloud Upload:** The React frontend uploads the raw binary asset (e.g., EV_Sales_India_2022_2025.xlsx) directly to Cloudinary's infrastructure, bypassing backend application bottlenecks.
4. **Asset Receipt:** Cloudinary stores the file, assigns an asset path (e.g., datapilot/datasets/ev_sales), and returns the cloudinary_public_id, cloudinary_url, file format, and byte size to the client.
5. **Metadata Registration:** The React client posts the asset reference metadata to FastAPI (POST /api/v1/datasets).
6. **Streaming Dataset Extraction:** FastAPI streams the remote file directly from Cloudinary via chunked HTTP byte streaming (leveraging Python's io.BytesIO and Pandas read_csv(storage_url) / read_excel()) to parse headers and rows in memory without persisting temporary files to the local API disk.
7. **Schema & Metadata Registration:** The engine infers data types, detects column headers, and populates a corresponding record in the PostgreSQL datasets table.
8. **Automated Quality Trigger:** The backend initiates asynchronous validation routines against the newly registered dataset state.

Cloudinary Innovation Statement

DataPilot AI uses Cloudinary as an active Asset, Evidence, and Provenance Layer rather than a simple image uploader. When DataPilot AI processes unstructured or structured raw files (CSV, XLSX, JSON, PDF), Cloudinary securely persists the immutable original inputs alongside any generated visual artifacts. When an end-user inspects an AI-generated analytical claim (e.g., "EV sales increased by 34.8%"), the platform constructs an audit trail that links directly to the

file's original Clouidnary Public ID, upload timestamp, and cryptographic file parameters. This architecture treats managed cloud storage as an integral component of scientific evidence chain auditing. This hybrid persistence layer provides a stable storage foundation for the platform's higher-level AI orchestration engine and sandboxed tool routines.

4. AI Orchestration, Intent Engine & Controlled Tool Layer

4.1 Intent Extraction & Structured Query Pipeline

Context & Strategic Importance

Natural language queries are inherently ambiguous, imprecise, and conversational. The AI Orchestrator serves as DataPilot AI's central cognition framework, converting raw human text into unambiguous, structured JSON execution specifications before running any data processing tasks.

Analytical Transformation

When a user submits a complex question—such as *"Compare EV sales in India from 2022 to 2025 and identify the fastest-growing manufacturers"*—the AI Orchestrator processes the query through an intent extraction pipeline. It isolates business metrics, dimensions, entity restrictions, time boundaries, target aggregations, and suggested chart types:

```
{
  "topic": "EV sales",
  "country": "India",
  "start_year": 2022,
  "end_year": 2025,
  "dimensions": [
    "manufacturer",
    "year"
  ],
  "metrics": [
    "sales",
    "growth_rate"
  ],
  "analysis": [
    "trend",
    "growth_comparison"
  ],
  "visualizations": [
    "line",
    "bar"
  ]
}
```

By decoupling language parsing from calculation execution, the system establishes a deterministic contract for downstream processing tools.

4.2 Dynamic Workflow Generation & Tool Execution Framework

Context & Strategic Importance

Static, rigid analytics pipelines fail when confronted with varying data tasks. Simple queries require basic statistical calculations, whereas cross-domain queries require multi-source ingestion, entity alignment, temporal normalization, and correlation routines. DataPilot AI resolves this by dynamically compiling custom execution graphs tailored to the extracted query intent.

Analytical Transformation

The platform constructs distinct execution plans across different query types:

Query A: Simple KPI Aggregation

User Query: "Calculate average product price."

1. Find Price Data: Retrieve dataset containing product pricing fields.
2. Validate Price Column: Scan for non-numeric characters or negative values.
3. Filter Invalid Records: Exclude non-numeric strings and values like 0\$.
4. Compute Mean: Calculate deterministic arithmetic average via Pandas.
5. Render KPI Card: Emit standard JSON payload for frontend single-value rendering.

Query B: Multi-Dataset Spatial & Temporal Correlation

User Query: "Compare rainfall and wheat production across Haryana districts from 2020 to 2025."

1. Fetch Rainfall Data: Query permitted India OGD API for Haryana district rainfall (2020–2025).
2. Fetch Crop Data: Retrieve district-level wheat yield production metrics.
3. Normalize Locations: Map district name variations (e.g., "Gurgoan" vs "Gurugram") to canonical names.
4. Normalize Years: Parse strings into standard ISO integer year values.
5. Outer Join Datasets: Merge datasets on district, year key pair.
6. Validate Coverage: Calculate join coverage percentage across district key sets.
7. Compute Pearson Correlation: Execute Pandas correlation matrix calculation (\$r\$).
8. Render Scatter Plot: Generate interactive scatter plot and trendline visualization configuration.

Query C: Time-Series Anomaly Detection

User Query: "Find unusual sales spikes."

1. Load Time-Series Data: Ingest date-indexed transaction records.
2. Validate Timestamp Alignments: Confirm missing date gaps and enforce daily frequency.
3. Aggregate Daily Sales: Sum transaction amounts grouped by calendar day.

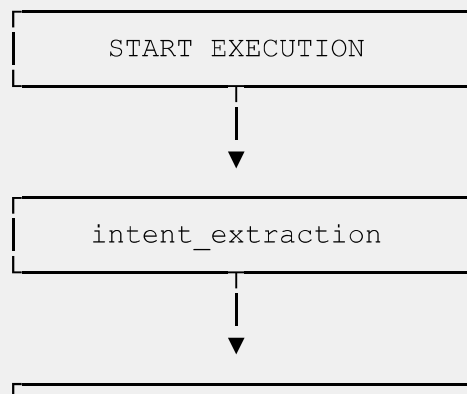
4. Run Anomaly Engine: Execute rolling z-score analysis ($|z| > 3.0$) to flag statistical outliers.
5. Render Annotated Chart: Emit line chart configuration highlighting anomaly dates. To enforce security and prevent arbitrary code execution, LLM agents are restricted to invoking predefined, sandboxed software tool primitives:

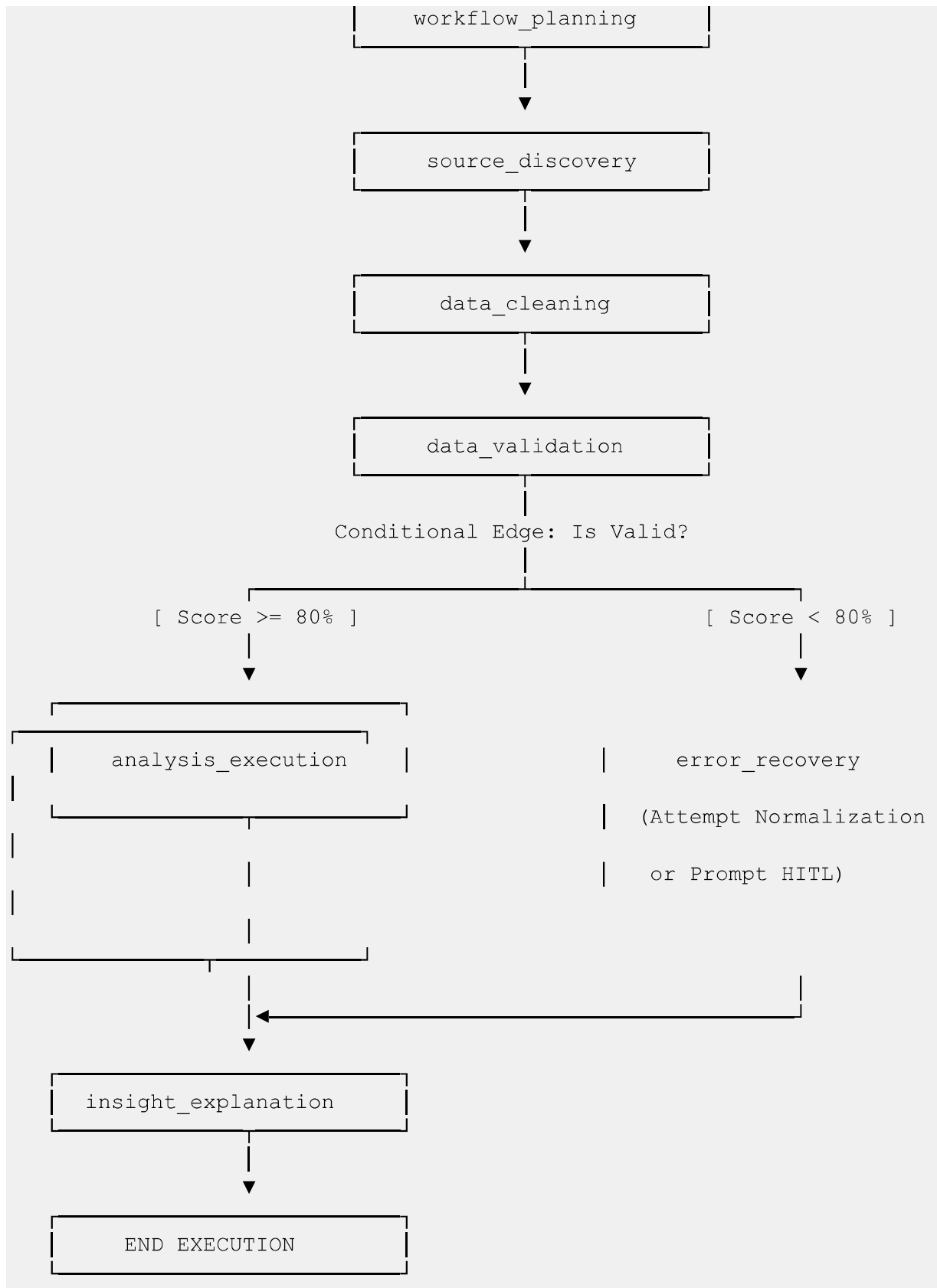
```
from pydantic import BaseModel, Field
from typing import List, Optional, Dict, Any

class ValidateDatasetInput(BaseModel):
    dataset_id: str = Field(..., description="UUID of dataset
registered in PostgreSQL")
    required_columns: List[str] = Field(..., description="Explicit
column names required for calculation")
    value_bounds: Optional[Dict[str, Dict[str, float]]] = Field(
        None, description="Min/Max acceptable numeric limits per
column, e.g., {'sales': {'min': 0.0}}")
    )
    check_duplicates: bool = Field(True, description="Enforce row
uniqueness check")

class RunAnalysisInput(BaseModel):
    workflow_id: str = Field(..., description="Active workflow
execution UUID")
    analysis_type: str = Field(..., description="Calculation
operation: 'CAGR', 'MEAN', 'PEARSON_CORRELATION', 'ANOMALY'")
    target_metric: str = Field(..., description="Primary column for
numeric calculation")
    group_by_dimensions: List[str] = Field(default=[],
description="Categorical columns for group aggregation")
    temporal_boundary: Optional[Dict[str, int]] = Field(None,
description="Start and end temporal bounds")
```

The system manages dynamic agent step flows using a LangGraph dynamic state transition machine:

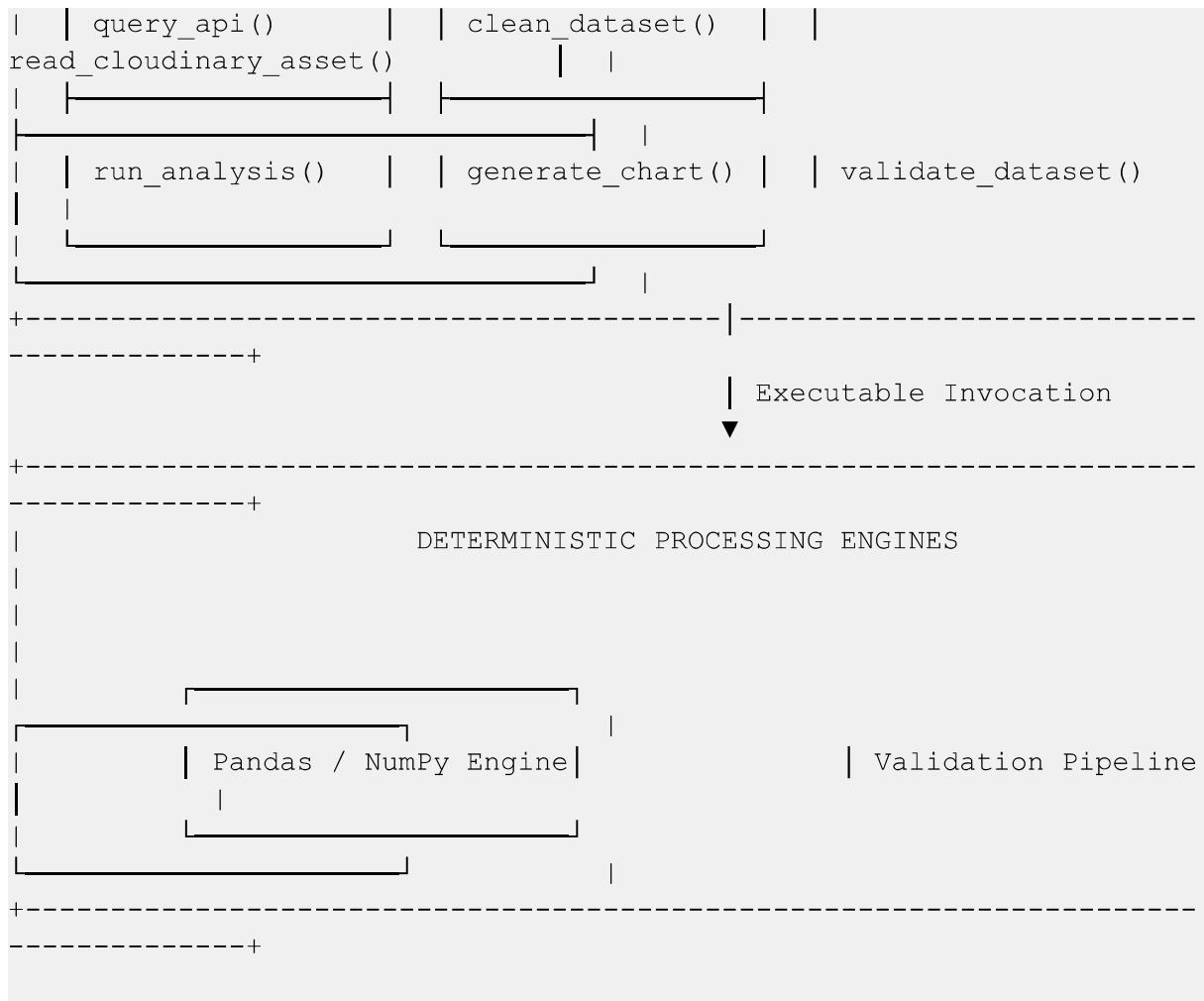




State transitions, conditional fallback routing, and execution timeouts are governed by explicit rules:

- **State Graph Nodes:** intent_extraction, workflow_planning, source_discovery, data_cleaning, data_validation, analysis_execution, insight_explanation, and error_recovery.
- **Conditional Edge Evaluation:** Following data_validation, if the quality score is $\geq 80.0\%$, the state transitions to analysis_execution. If the quality score falls below 80.0% , the graph routes to error_recovery to attempt secondary entity normalization or trigger Human-in-the-Loop (HITL) confirmation.
- **Execution Timeout Parameters:** Every individual engine node is capped at a 30-second execution deadline. If a node times out, the machine aborts the execution path, logs an exception in workflow_steps.error, and safe-fails to a user warning state.





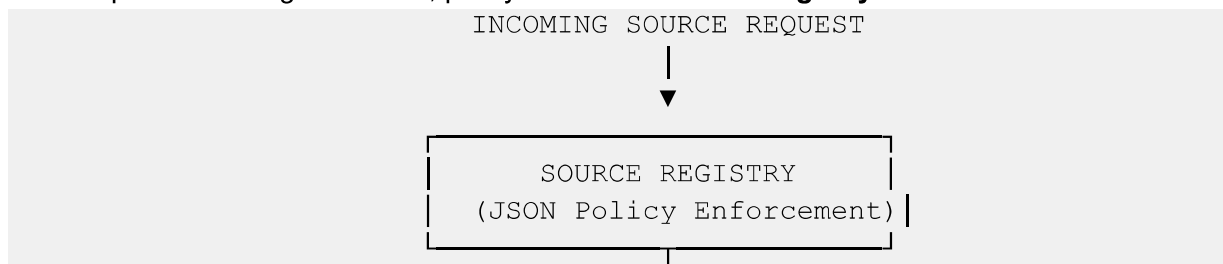
This controlled environment enforces a clean separation between non-deterministic planning and deterministic data processing.

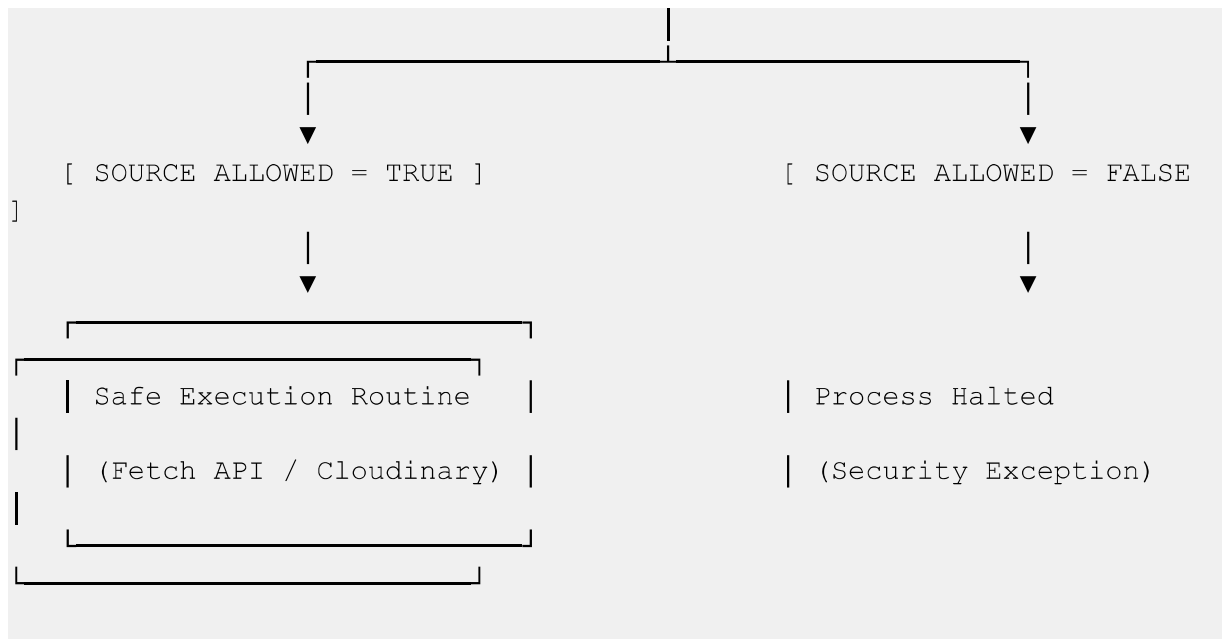
5. Deterministic Execution Pipelines & Anti-Hallucination Mechanics

5.1 Permitted Source Registry & Data Collection Engine

Context & Strategic Importance

Ingesting unverified external data introduces severe risks, including hallucinated source claims, licensing non-compliance, and data corruption. DataPilot AI mitigates these risks by routing all data acquisition through a central, policy-enforced **Source Registry**.





Analytical Transformation

The Source Registry restricts data fetching logic to administrative configurations. Every source configuration tracks governance metadata alongside explicit data freshness parameters:

```
{
  "source_id": "ogd_india",
  "name": "India Open Government Data",
  "type": "api",
  "allowed": true,
  "license": "Government Open Data License",
  "domains": [
    "agriculture",
    "economy",
    "health"
  ],
  "freshness_metadata": {
    "last_updated": "2026-09-18T00:00:00Z",
    "retrieved_at": "2026-09-30T08:30:00Z",
    "coverage_period": "2020-2025",
    "stale_threshold_days": 180
  }
}
```

During execution, the Data Collection Engine compares retrieved_at timestamps against stale_threshold_days. If an asset exceeds the threshold, the system triggers an automated refresh warning in the UI while verifying that target APIs match permitted domain rules and active license flags. Server-side secrets, authentication headers, and API keys are injected

exclusively within the backend boundary, preventing credential exposure to client environments or LLM prompts.

5.2 Deterministic Data Processing & Cleaning Engine

Context & Strategic Importance

Raw real-world datasets are inherently messy—containing mixed date formats, inconsistent entity strings, conflicting measurement units, duplicate rows, and invalid values. Allowing raw data to pass directly to analytical routines produces incorrect outputs and system errors. DataPilot AI addresses this by running all raw inputs through a deterministic Pandas/NumPy processing engine before analysis occurs.

Analytical Transformation

The data processing engine performs standardized normalization and cleaning routines:

```
RAW DATASET INGESTION
├─ String Entities: ["Tesla", "TESLA", "Tesla Inc.", "tesla"] →
Entity Normalization → "Tesla"
├─ Timestamp Strings: ["2024", "2024-01-01", "01/01/2024"] →
ISO Normalization → 2024-01-01
├─ Numeric Strings: ["₹5 lakh", "$500,000", "500000"] →
Coercion Engine → 500000.00
├─ Data Types: [Mixed Float, Int, String Formats] →
Type Enforcement → Strict Types
```

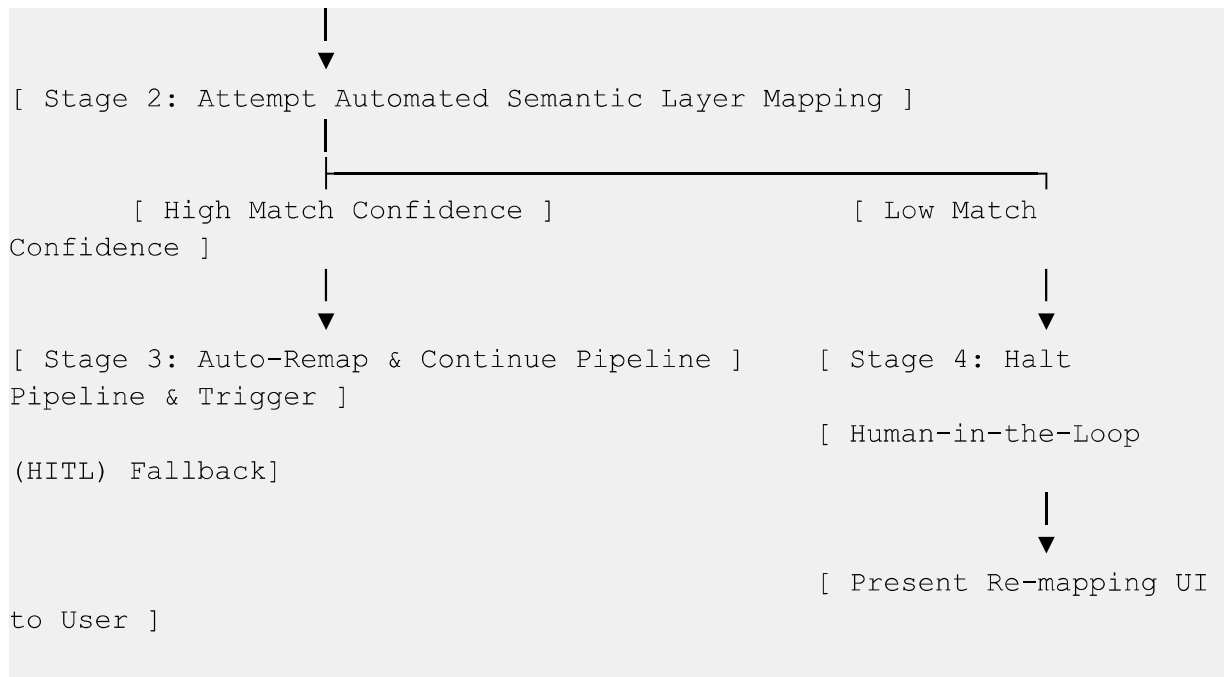
To handle bad data inputs safely without crashing execution workflows, the engine applies defensive rules:

- **Entity Resolution:** Unifies string variants (e.g., mapping "Tesla", "TESLA", and "Tesla Inc." to a canonical "Tesla" string) using deterministic normalization logic.
- **Temporal Harmonization:** Standardizes mixed date strings (e.g., "2024", "2024-01-01", "01/01/2024") into consistent ISO datetime fields.
- **Numeric Coercion:** Detects currency strings or textual numbers (e.g., "₹5 lakh") within numeric columns, extracts raw numeric values, and coerces the column to standard float/integer types.
- **Out-of-Bounds Handling:** Identifies impossible metrics (e.g., an EV sales volume of -500) and isolates the offending rows, flagging them for validation review instead of allowing them to skew statistical results.

Schema Drift Detection & Human-in-the-Loop (HITL) Fallback

When ingested data schemas alter unexpectedly (e.g., a source renames column revenue to total_revenue), the cleaning engine executes a controlled multi-stage drift protocol:

```
[ Ingested Schema Alteration Detected ]
      |
      ▼
[ Stage 1: Generate System Schema Warning ]
```



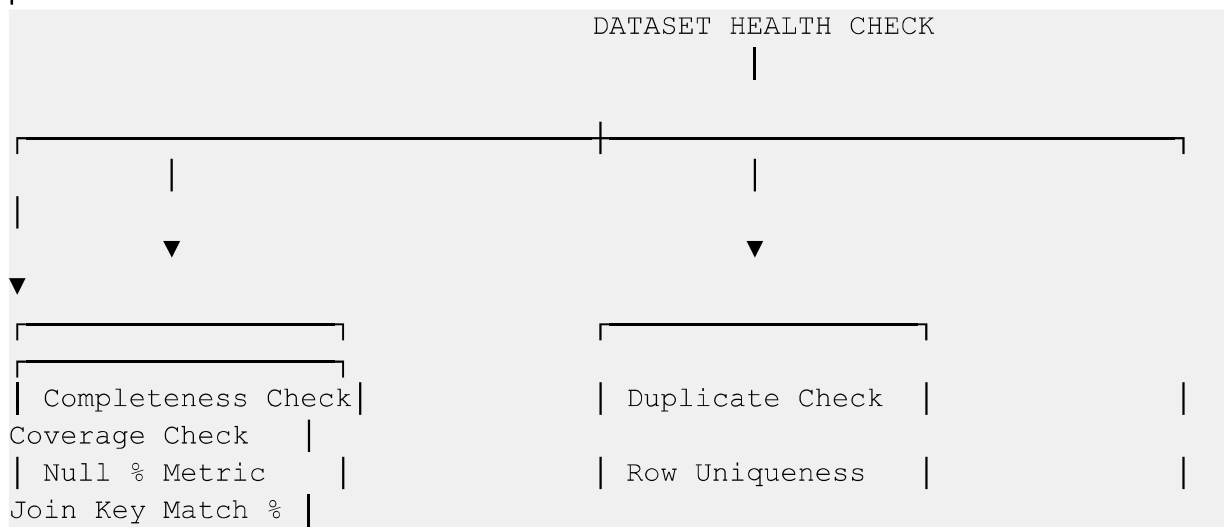
5.3 Automated Data Validation Engine & Quality Scoring

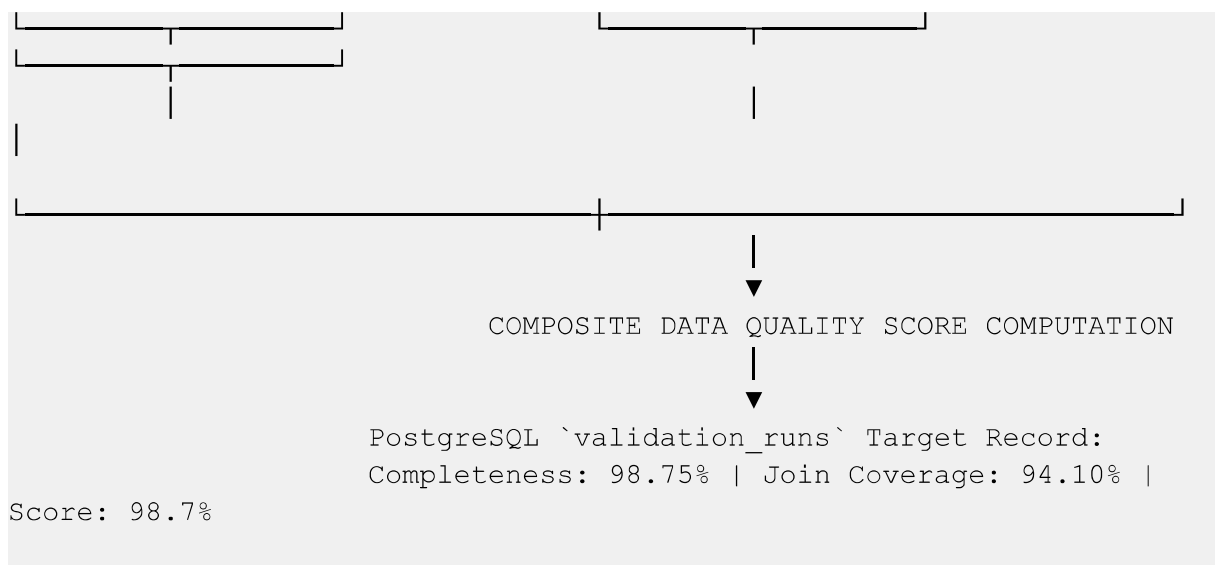
Context & Strategic Importance

Rather than making blind assumptions about data integrity, DataPilot AI evaluates every processed dataset using an automated validation engine. This step runs explicit quality checks to calculate a transparent, quantitative Data Confidence metric before downstream analytics begin.

Analytical Transformation

The Validation Engine runs health checks across individual tables and joined multi-dataset key pairs:





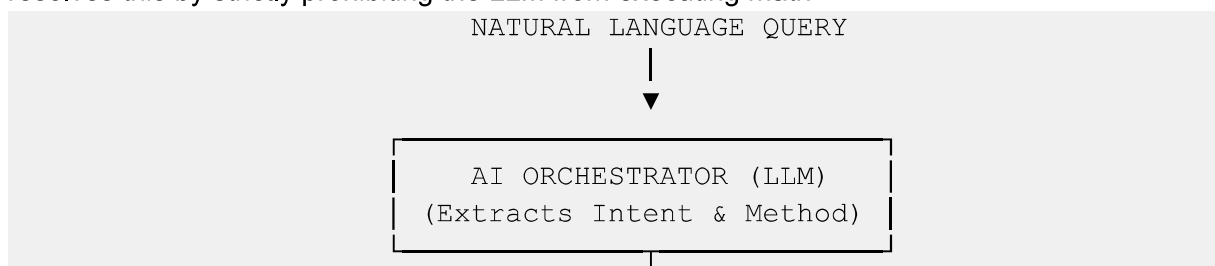
Data health is quantified using a multi-factor weighting formula: $\text{Data Quality Score} = w_1 \cdot \text{Completeness} + w_2 \cdot (1 - \text{Duplicate Rate}) + w_3 \cdot \text{Schema Validity} + w_4 \cdot \text{Join Coverage}$ Where:

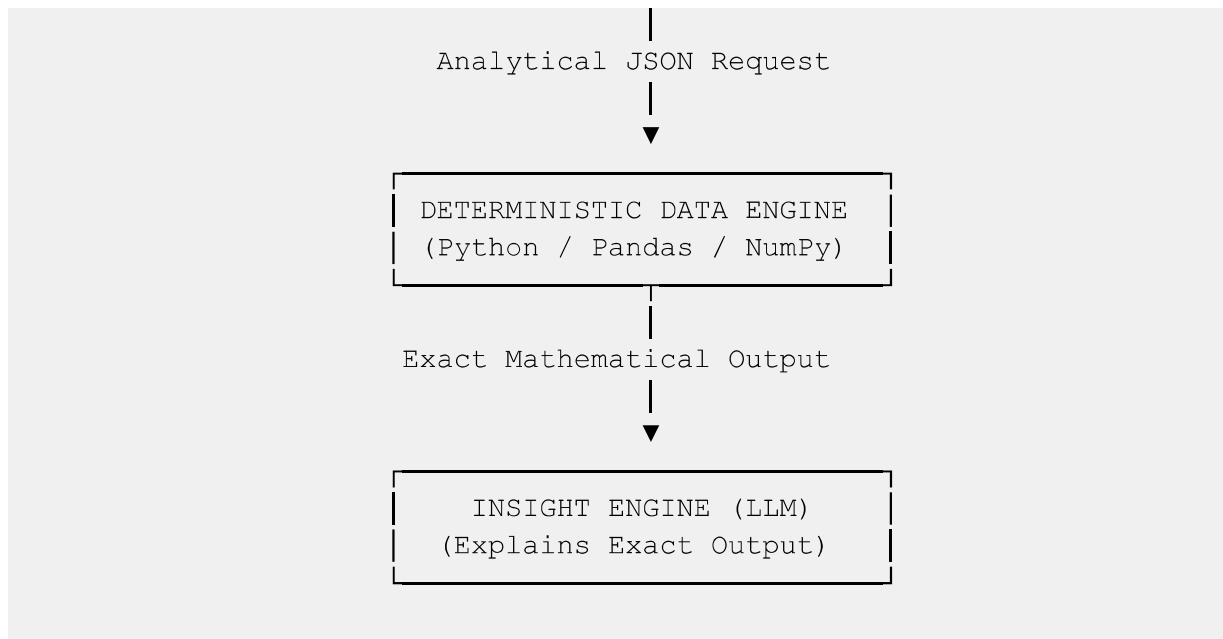
- $\text{Completeness} = \left(1 - \frac{\text{Null Records}}{\text{Total Records}}\right)$ (Default weight $w_1 = 0.35$)
 - $\text{Duplicate Rate} = \left(\frac{\text{Duplicate Key Rows}}{\text{Total Records}}\right)$ (Default weight $w_2 = 0.25$)
 - $\text{Schema Validity} \in \{0.0, 1.0\}$ indicating structural match (Default weight $w_3 = 0.20$)
 - $\text{Join Coverage} = \left(\frac{|\text{Keys}_{\text{DatasetA}} \cap \text{Keys}_{\text{DatasetB}}|}{|\text{Keys}_{\text{DatasetA}} \cup \text{Keys}_{\text{DatasetB}}|}\right)$ evaluating relational overlap across multi-dataset joins (Default weight $w_4 = 0.20$)
- The calculated metrics are stored in PostgreSQL table `validation_runs`, supplying the frontend UI with quantitative health indicators (e.g., 98.7% Quality Score, 94.1% Join Coverage).

5.4 Deterministic Analysis Engine & Strict LLM Separation

Context & Strategic Importance

A fundamental design flaw in conversational AI is relying on Language Models to execute numerical computations. LLMs construct text responses based on probabilistic token predictions, making them fundamentally unreliable for mathematical calculations. DataPilot AI resolves this by strictly prohibiting the LLM from executing math.





Analytical Transformation

The mathematical execution pipeline follows a strict, single-direction path:

1. **Operation Determination:** The AI Orchestrator identifies the mathematical operation needed to answer the user query (e.g., calculating percentage growth, mean values, Pearson correlations, or aggregated sums).
2. **Engine Execution:** The analytical request is routed to the deterministic Python engine (Pandas/NumPy). The engine performs the calculations on the verified dataset and outputs exact numerical values.
3. **Narrative Explanation:** The calculated result (e.g., 34.8% growth) is passed back to the Insight Engine LLM. The LLM's role is restricted to translating the exact output into a clear, natural language explanation without altering any values. The system incorporates explicit edge-case protocols:
 - **Handling Ambiguous Queries:** When a query lacks required parameters, the Orchestrator halts execution and returns a structured JSON clarification response:

```
{
  "status": "NEEDS_CLARIFICATION",
  "error_code": "AMBIGUOUS_QUERY_BOUNDARIES",
  "message": "Your query requests sales for Delhi but omits the
temporal boundary and metric definition.",
  "clarification_prompt": {
    "parameter_missing": ["time_period", "metric_type"],
    "suggested_options": {
      "time_period": ["2022-2025", "FY2024", "Last 12 Months"],
      "metric_type": ["Gross Revenue", "Net Volume (Units)"]
    }
  }
}
```

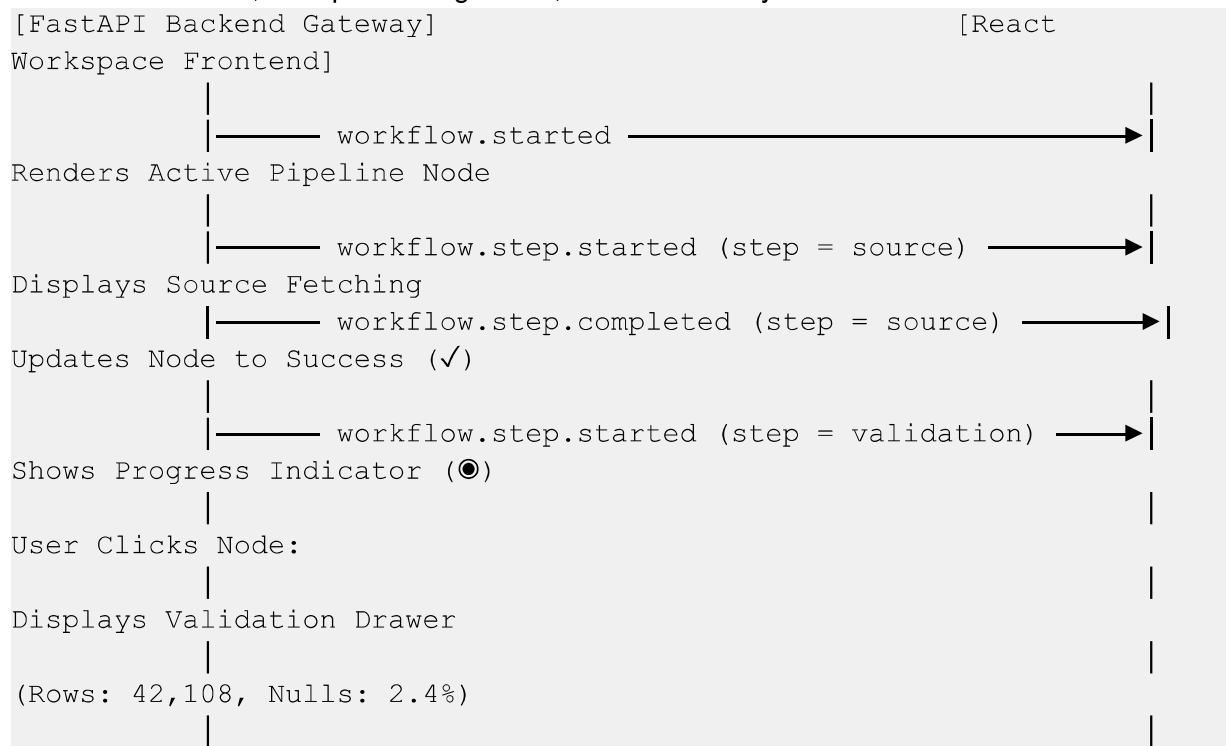
- **Handling Insufficient Data:** If a user requests a causal explanation (e.g., *"Why did sales fall?"*) but the underlying dataset contains only date and sales fields, the system refuses to speculate. It notifies the user: *"I can identify when sales decreased (e.g., -14.2% in Q3), but the available dataset lacks causal variables (e.g., marketing spend, unit pricing, or inventory levels) to determine why."*
- **Correlation vs. Causation Guardrails:** When statistical correlation logic runs across two variables (e.g., rainfall and crop output), system prompts strictly constrain the Insight Engine:
System Instruction: "The calculated Pearson coefficient ($r = 0.78$) demonstrates a positive statistical association. You are strictly forbidden from making causal claims such as 'Rainfall caused crop yields to rise'. Output must explicitly state that the relationship represents a statistical correlation." This multi-stage deterministic framework yields fully auditable data flows that integrate directly into the frontend user interface.

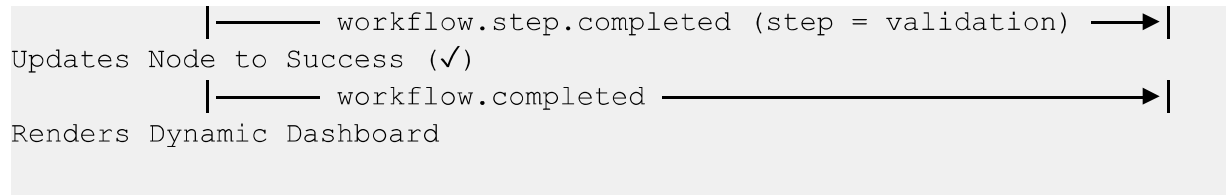
6. Visualization, Intelligence & Auditability Model

6.1 Interactive Frontend Architecture & Real-Time Updates

Context & Strategic Importance

Modern enterprise data workspaces require real-time visibility into active processes. Static loading indicators are insufficient for multi-stage workflows. DataPilot AI provides an interactive workspace built with React, TypeScript, and React Flow, giving users full real-time visibility into workflow execution, data processing states, and visual analytics.





Analytical Transformation

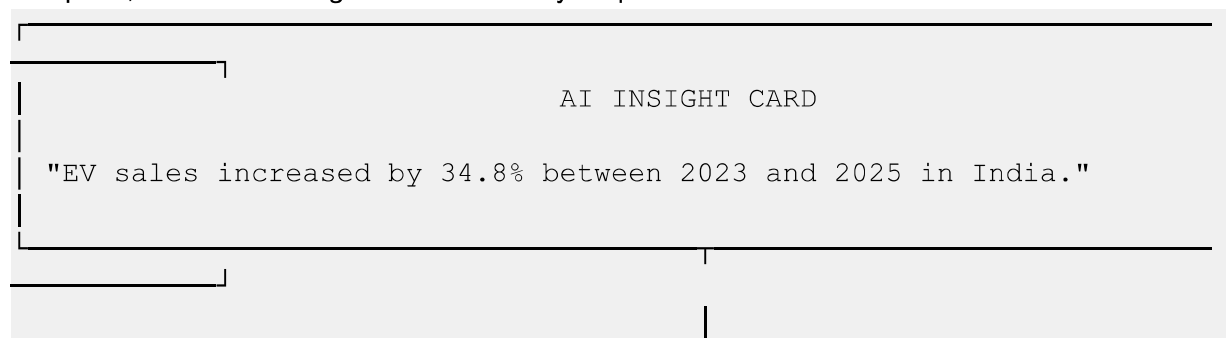
The frontend architecture relies on two key rendering modules:

- **Real-Time Streaming via Server-Sent Events (SSE):** The backend streams workflow state changes using SSE endpoints (`/api/v1/workflows/{id}/stream`). Transmitted events trigger real-time updates across the client interface:
- `workflow.started`: Initializes the workspace view and constructs the initial pipeline node layout.
- `workflow.step.started`: Highlights the active step node in the dynamic interface (e.g., `step = validation`).
- `workflow.step.completed`: Updates the node status badge to complete, displaying processing times and metrics.
- `workflow.completed`: Finalizes workflow rendering and loads the final dashboard view.
- **Interactive Workflow Graph (React Flow):** The client renders dynamic execution graphs using React Flow. Users can click individual workflow nodes to open detailed inspection panels:
- *Source Node Click*: Displays target source APIs, license metadata, and raw retrieval parameters.
- *Validation Node Click*: Displays total rows evaluated (e.g., 42,108), null percentages (2.4%), duplicate counts (37), and schema evaluation flags.
- *Analysis Node Click*: Displays exact statistical execution logic, formulas, and execution runtimes.

6.2 Complete Evidence Engine & Lineage Traceability

Context & Strategic Importance

Enterprise analytics demands complete accountability. Decision-makers must be able to trace every chart metric and AI-generated insight back to its underlying raw data, cleaning steps, and calculation formulas. DataPilot AI implements a dedicated **Evidence Engine** that generates complete, traceable lineage chains for every output.



[View Evidence]

EVIDENCE & LINEAGE TRAIL

1. COMPUTED INSIGHT

→ "EV sales increased by 34.8%"

2. CALCULATION FORMULA

→ ((sales_2025 - sales_2023) / sales_2023) * 100

3. RAW INPUT VALUES

→ 2023: 1,130,000 | 2025: 1,523,400

4. TRANSFORMATIONS

→ Duplicate removal; Manufacturer entity mapping

5. DATA VALIDATION

→ 98.7% Data Quality Score (0 Invalid Values)

6. REGISTERED DATASET

→ EV_Sales_India_2022_2025.xlsx

7. CLOUDINARY ASSET

→ Public ID: datapilot/datasets/ev_sales

8. ORIGINAL SOURCE

→ Permitted API: Open Government Data India

Analytical Transformation

The Evidence Engine exposes specific inspection endpoints (GET /api/v1/workflows/{id}/evidence) that yield structured payload traces:

```
{
  "workflow_id": "w_456",
  "execution_runtime_ms": 1245,
  "insight": "EV sales grew 34.8% between 2023 and 2025",
  "calculation": {
    "formula": "(sales_2025 - sales_2023) / sales_2023 * 100",
    "inputs": {
      "sales_2023": 1130000,
      "sales_2025": 1523400
    },
  },
  "result": 34.814,
  "rows_analyzed": 152340,
  "record_offset_range": "0:152340"
```

```

},
"transformations": [
  "Filtered range to year >= 2022 AND year <= 2025",
  "Purged 42 duplicate records",
  "Normalized manufacturer entity variations to canonical names"
],
"validation": {
  "quality_score": 98.7,
  "completeness": 98.9,
  "invalid_records": 0,
  "join_coverage": 100.0
},
"dataset": {
  "name": "EV_Sales_India_2022_2025.xlsx",
  "sha256_checksum":
"e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855",
  "cloudinary_public_id": "datapilot/datasets/ev_sales_123",
  "cloudinary_url":
"https://res.cloudinary.com/demo/raw/upload/v1/datapilot/datasets/ev_s
ales_123.xlsx"
},
"source": {
  "source_id": "ogd_india",
  "name": "India Open Government Data",
  "license": "Government Open Data License"
}
}

```

This lineage structure ensures every analytical claim is backed by complete, verifiable mathematical and infrastructural evidence.

6.3 Security Architecture, Governance & Defensive Engineering

Context & Strategic Importance

Operating an AI-orchestrated analytics platform within enterprise environments requires multi-layered defensive security. Isolating user project workspaces, managing server credentials securely, enforcing sandboxed tool execution, and maintaining strict governance standards are required to protect data integrity and prevent unauthorized access.

Analytical Transformation

DataPilot AI incorporates multi-tenant security controls across its application layers:

- **Multi-Tenant Workspace Isolation:** Every query, dataset reference, workflow execution, and evidence record is mapped strictly to a valid `project_id` and `user_id`. Backend API authorization middleware blocks cross-project data access.

- **Credential & Secret Protection:** Third-party credentials, cloud connection tokens, and Cloudinary secrets are managed exclusively on the backend server. The React client framework receives only short-lived, signed upload parameters.
- **Sandboxed Tool Invocation:** The AI Orchestrator cannot run system commands or execute arbitrary code. It interacts with data solely by invoking explicit, sandboxed tool functions governed by strict input schemas.
- **Read-Only Database Scopes:** SQL tool execution wrappers enforce read-only transaction limits, blocking destructive operations (DROP, INSERT, UPDATE, DELETE) at the database driver level.
- **Schema Drift HITL Fallback:** The processing framework monitors dataset header structures during ingestion. If schema alterations occur without clear semantic mapping confidence, execution halts and triggers human confirmation via the UI. To summarize the system's core technical mechanisms, the defensive matrix below highlights the architectural solutions engineered to address common platform inquiries: | Core Inquiry / Scenario | Technical Architecture & Defensive Mechanism || ----- | ----- || **"How do you prevent AI hallucinations?"** | Non-deterministic LLMs are strictly prohibited from performing numerical or statistical calculations. Math logic is delegated entirely to the deterministic Python engine (Pandas/NumPy). The LLM functions solely as an orchestrator, query planner, and textual explainer. || **"Why use Cloudinary in a data platform?"** | Cloudinary serves as the platform's immutable Asset and Evidence Layer. It manages raw dataset uploads (.csv, .xlsx, .json, .pdf), evidence media, and generated visual artifacts via signed transfers. This offloads binary file storage from backend application servers and links every insight directly to an immutable source asset. || **"How is DataPilot AI different from ChatGPT?"** | ChatGPT generates single-pass conversational text responses. DataPilot AI dynamically builds, validates, and executes end-to-end data engineering pipelines—ingesting verified datasets, applying cleaning routines, validating data health, running calculations, rendering charts, and generating auditable evidence trails. || **"Where exactly is AI applied in the system?"** | AI is concentrated within the Orchestration and Cognition Tier. It parses natural-language queries into structured JSON blueprints, plans dynamic execution steps, selects required tools, and translates verified calculation outputs into clear, natural language explanations. || **"What happens if input data is corrupted or low quality?"** | Processed datasets must pass through the Automated Validation Engine before analysis occurs. The engine scans for structural nulls, row duplicates, invalid metrics, and type mismatches, returning explicit quality scores (e.g., 98.7% Quality Score). If severe errors are present, the system alerts the user instead of presenting unverified metrics. |

DataPilot AI converts arbitrary, natural-language information requests into transparent, executable, and fully auditable data workflows—enforcing deterministic processing execution, automated quality validation, and complete evidence traceability across every generated insight.